Práctica 2: Visión artificial y aprendizaje

Objetivos

- Comprender el funcionamiento general de las redes neuronales artificiales como técnicas de aprendizaje supervisado, y en concreto los perceptrones multicapa y las redes neuronales convolucionales.
- Entender cómo podemos utilizar la información de los píxeles de una imagen en visión artificial para distinguir lo que representa esa imagen.
- Conocer la librería Keras para implementar redes neuronales de forma rápida y efectiva.
- Utilizar redes neuronales para resolver un problema real de clasificación.
- Entender el papel de cada uno de los principales parámetros que configuran una red neuronal, y aprender a ajustarlos de forma efectiva y eficiente para maximizar su tasa de acierto a la vez que se minimiza el tiempo de entrenamiento.
- Utilizar Matplotlib para mostrar resultados de forma visual.
- Acelerar el flujo de trabajo utilizando *scripts* que generen de forma automática las pruebas para el ajuste de parámetros y los contenidos para el informe (resultados de clasificación, tablas, gráficas).

1) INTRODUCCIÓN AL PROBLEMA A RESOLVER Y AL ENTORNO DE TRABAJO. NORMAS DE ENTREGA Y EVALUACIÓN....3

Introducción....3
Evaluación....3
Entorno de trabajo....3
Documentación....4
Bibliografía....4
Formato y fecha de entrega....4

2) TOMA DE CONTACTO CON CIFAR 10 Y LAS GRÁFICAS DE MATPLOTLIB....5

Trabajando con las imágenes de Cifar10....5
Gráficas de los resultados con Matplotlib....6

3) PRIMERA PARTE: APRENDER A CONFIGURAR REDES NEURONALES CON MLPS DE KERAS....7

3.1 Definir, entrenar y evaluar un MLP con Keras, e identificar el número adecuado de épocas para evitar sobre- y subentrenamiento....7
3.2 Algunos parámetros básicos de un MLP....8
3.3 FUNCIONES DE ACTIVACIÓN Y PROFUNDIDAD DE LA RED....8
3.4 Técnicas de normalización: Batch Normalization....9
3.5 Técnicas de regularización: Dropout....10

4) SEGUNDA PARTE: DESARROLLAR UN MODELO EFICIENTE CON CNN DE KERAS....10

4.1 Definir, entrenar y evaluar un CNN sencillo con Keras....10
4.2 Tarea final: desarrollar una CNN para Cifar10 con optimización libre....11

5) CONSIDERACIONES FINALES....11

1) Introducción al problema a resolver y al entorno de trabajo. Normas de entrega y evaluación.

Introducción

En esta práctica trabajarás con técnicas de aprendizaje supervisado aplicadas al reconocimiento automático de imágenes en problemas reales de dificultad moderada y alta. En nuestro caso, clasificar diez tipos de vehículos y animales de la base de datos Cifar10 (https://www.cs.toronto.edu/~kriz/cifar.html). Utilizarás la librería Keras (https://keras.io/) de TensorFlow, que facilita configurar modelos de redes neuronales para problemas de clasificación y regresión.

La práctica se divide en dos partes. La primera está centrada en perceptrones multi-capa (o MLP de multi-layer perceptron en inglés) en las que veremos conceptos básicos del entrenamiento de redes neuronales realizando ejercicios guiados relativamente sencillos, para que te familiarices con los principales parámetros y aprendas a ajustarlos de forma eficiente y eficaz. La segunda parte está centrada en redes neuronales convolucionales (CNN de convolutional neural networks), que son mucho más efectivas para el procesamiento de imágenes, y en la que realizarás una tarea más compleja y libre: desarrollar y optimizar un modelo al completo para maximizar su tasa de acierto, en la que demostrarás las capacidades adquiridas.

Evaluación

Durante las sesiones de prácticas el o la docente realizará preguntas de seguimientos individualmente, que comprenderán el 40% de la nota final, basado sobre todo en la realización de los ejercicios y el desarrollo inicial de la tarea de optimización final. El 60% restante se evaluará en la documentación que se entregue al final, valorando más las competencias de análisis y exploración que demuestre el estudiante antes que la calidad final del modelo presentado (es decir, presentar un modelo muy bueno pero sin demostrar cómo se ha llegado hasta él obtendrá menos nota que mostrar el desarrollo fundamentado de un modelo menos eficiente, porque lo que se valorará ante todo es que hayas adquirido conocimientos útiles).

Entorno de trabajo

Trabajaremos con Python en Linux. Python es un lenguaje interpretado multiplataforma muy extendido, se puede usar en todos los sistemas operativos de uso general. Puedes elegir el entorno de trabajo que prefieras (tanto sistema operativo como IDE), pero debes asegurarte de que tu código fuente funciona correctamente en Linux, en los PC de los laboratorios, porque ese es el entorno en que se evaluará la entrega.

Si no tienes experiencia con Python, se recomienda no usar Visual Studio y utilizar en su lugar el IDE Thonny (https://thonny.org/), ya instalado en los laboratorios, que da mejores ayudas a programadores de Python nóveles, y además tiene la ventaja de utilizar su propia instalación de Python, independiente de la del sistema operativo, de forma que no genera conflictos y facilita la instalación de paquetes como Tensorflow y Keras. Para instalar un paquete en Thonny ve a Herramientas → Gestionar paquetes, y ahí busca el paquete que desees instalar.

Además de Keras, en general siempre trabajaremos con Numpy (https://numpy.org/) para la manipulación eficiente de arrays, y alguna de las facilidades que nos provee Pandas (https://pandas.pydata.org/), y para el dibujado de imágenes y gráficas en pantalla utilizaremos Matplotlib (https://matplotlib.org/). También podrás utilizar cualquier librería de Python que se pueda instalar en vuestro Thonny o usando `pip --user install <librería>`, pero deberás consultar primero con tu docente. Como alternativa que no requiere instalación, tendrás la opción de utilizar Google Colab para realizar la práctica.

Si tu código no puede ejecutarse en los laboratorios o en Google Colab, no será evaluado, salvo acuerdo previo con el docente de tu grupo de prácticas.

Documentación

La documentación es la parte más **importante** de la práctica. Tu documentación incluirá una sección para cada ejercicio o tarea, donde pondrás cualquier apunte que consideres relevante sobre su desarrollo, las pruebas que hayas realizado, las mejoras que hayas implementado y las conclusiones extraídas de esos experimentos. Es importante que concretes y sintetices (que vayas al grano y que resumas).

A menudo, la documentación de un experimento puede ser un poco confusa. Por ello, si tienes dudas sobre cómo presentar tus resultados, **consulta con tu profesor/a durante las horas de clase o en una tutoría**. Evita copiar información de chatbots o de fuentes que no entiendes bien. La clave no es la cantidad de texto o de gráficos, sino **la calidad y la claridad** de tu trabajo y que demuestre aprendizajes útiles. De nada sirve llenar páginas con tablas y gráficas repetitivas. Lo que se valora es que la documentación refleje que has comprendido el experimento y has alcanzado los objetivos de aprendizaje. Tu documentación debe mostrar que has aprendido lo que se te pide, no que has realizado muchos experimentos monótonos o que has usado lenguaje complicado sin entenderlo. ¡Consulta a tu profesor si tienes dudas sobre esto!

Bibliografía

La documentación debe incluir al final una sección de bibliografía con todas las referencias que utilices, o bien, estas referencias deben enlazarse en el texto donde se usen (entre paréntesis o como nota al pie). Si utilizas una sección de bibliografía, cada referencia en esa sección debe aparecer citada en el texto, en los apartados en que se ha usado. Una bibliografía mínima puede consistir solo en los apuntes de clase (aunque recomendamos que investigues más por tu cuenta). **La ausencia de bibliografía podrá penalizar hasta 1 punto**. Este requerimiento no significa que debas buscar algo que citar para rellenar, porque una buena bibliografía no da puntos, solo te permite aprender más deprisa y hacer mejor el trabajo. Si no has usado ninguna referencia externa, no debes perder el tiempo en inventarte una bibliografía. **También será motivo de penalización incluir referencias que no se usen en el texto**.

Las referencias a vídeos de plataformas como Youtube deben incluir un breve resumen de la información que se ha extraído de esa fuente. Las consultas a ChatGPT, Claude, Bing y otros chatbots basados en grandes modelos de lenguaje son también parte de la bibliografía y deben ser referenciados. Si copias sus respuestas breves textualmente, debes marcarlas como una cita, entre comillas. Si los usas para entender conceptos o desarrollar explicaciones, en una conversación con el chatbot, no debes copiar la conversación entera en mitad de la documentación, pero puedes añadirla como un anexo al final (y citas la conversación desde el texto en que la usas), aunque será suficiente con que indiques en cada parte del texto en qué cuestiones te has apoyado en el chatbot.

Formato y fecha de entrega

Esta práctica tiene un hito intermedio, con fecha tope el 29 de noviembre de 2026, y una entrega final, hasta el 23 de diciembre de 2026 a las 23:55h. En el hito intermedio deberás entregar como mínimo los ejercicios 1 a 5 resulto, tanto implementación como documentación (que puede ser un borrador, lo importante es que incluya las conclusiones que has ido sacando al realizar los ejercicios). Esta entrega intermedia no se evalúa, pero la no entrega de este hito, o una entrega incompleta (en cuanto a la implementación), tendrá una penalización de hasta el 20% de la nota final de la práctica. En la entrega final deberás entregar la práctica completa, ahora sí obligatoriamente con toda la documentación bien redactada, y tendrás opción de mejorar lo que hubieras entregado en el hito intermedio.

Para permitir la evaluación de tu entrega, deberás crear un módulo (una función) en tu archivo Python para cada tarea de la práctica, que llamará a todas las funciones implicadas en completar esa tarea. La idea es que los profesores podamos ejecutar solo la parte que queramos de tu código, sin tener que esperar a que se entrenen todos los clasificadores de todas las tareas. Estos módulos se podrán llamar individualmente en la última sección de tu archivo Python, utilizando el condicional `__name__ == "__main__"`. Puedes ver cómo se

hace en la plantilla de código funcional que se adjunta en Moodle y que ya trae resueltos los dos primeros ejercicios.

Puede que algunas tareas requieran varios módulos, o que reúsen módulos de otras tareas con distintos parámetros (en el código de ejemplo, el módulo probar_MLP se usa para declarar, compilar, entrenar y evaluar un modelo de MLP, definido por sus argumentos). Si los nombres de tus funciones son ambiguos, utiliza comentarios para dejar claro qué hace cada módulo. En el momento de la entrega, **deja comentados (para que no se ejecuten) todos los módulos menos el de la última tarea**.

Las prácticas son individuales, no se pueden hacer en grupos y no se puede reusar código o texto de otros estudiantes ni de otras fuentes sin referenciarlo. La entrega se realizará **a través de Moodle**, y debe consistir en dos ficheros, **un fichero .pdf con la documentación y otro fichero .py con todo el código Python**, tanto las implementaciones de los algoritmos requeridos como el código utilizado para producir todos los resultados que muestres en la documentación (gráficas, tablas, etc.). No se puede entregar la práctica en varios ficheros de Python. Si deseas programar en varios archivos por el motivo que sea, asegúrate de juntarlo todo en un único archivo .py y comprobar que todo funciona correctamente antes de la entrega. Los dos archivos de la entrega se deben llamar con el nombre completo del estudiante, usando '_' como separador entre palabras (p.ej. Juan_Carlos_Sánchez-Arjona_de_los_Rios.py y .pdf). En el caso de que creas necesario adjuntar algún otro archivo (una hoja de cálculo con resultados, un archivo de log, etc.) consulta primero a tu profesor/a.

2) Toma de contacto con Cifar 10 y las gráficas de Matplotlib

Trabajando con las imágenes de Cifar10

Vamos a cargar la base de datos de Cifar10 y familiarizarnos con la manipulación de esas imágenes. Hay muchas maneras de cargar las imágenes de Cifar10 en un programa Python. Nosotros utilizaremos la función que nos facilita Keras (https://keras.io/api/datasets/cifar10/). Para ello primero hay que importar esa librería (e instalarla si no está instalada). Al ejecutar ese `import` algunos veréis una serie de mensajes de advertencia de TensorFlow, quejándose de que no encuentra soporte para usar CUDA en la tarjeta gráfica. Quienes tengan una tarjeta NVIDIA pueden instalar el soporte para CUDA y conseguir entrenamientos de modelos de aprendizaje mucho más rápidos gracias a la paralelización, pero no es necesario en esta práctica y no afecta a la calificación. Si queremos eliminar esos mensajes de error por comodidad, podemos utilizar el siguiente código al principio de nuestro archivo .py:

```
import logging, os
logging.disable(logging.WARNING)
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
```

Ahora podemos cargar las imágenes de Cifar10 y ver en qué consisten:

```
import keras

(x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data()
print(x_train.shape, x_train.dtype)
print(y_train.shape, y_train.dtype)
print(x_test.shape, x_test.dtype)
```

```
print(y_test.shape, y_test.dtype)
```

Esos print nos dicen que `x_train` es un array de Numpy de 50 000 elementos (en este caso observaciones, o *samples* en inglés) que a su vez son arrays de 32x32x3 `uint8` (unsigned int de 8 bits, números de 0 a 255), es decir, un array de 50 000 imágenes en color de 32x32 píxeles. Cada uno de esos 32x32x3=3072 bytes de una imagen es una característica de una observación. `Y_train` es otro array de 50 000 elementos `uint8` que contiene los 50 000 números asociados a cada categoría (0: avión; 1: coche; 2: pájaro, etc.), y que son los valores que querremos que nos devuelva nuestro MLP para cada imagen cuando esté bien entrenado. Y los arrays `x_test` e `y_test` son iguales pero contienen 10 000 observaciones cada uno.

Podemos mostrar en pantalla las imágenes que hay en `X_train` e `X_test` utilizando la función `imshow` de Matplotlib. El siguiente código mostrará tres imágenes al azar del conjunto de entrenamiento:

```
import matplotlib.pyplot as plt

def show_image(imagen, titulo):
    plt.figure()
    plt.suptitle(titulo)
    plt.imshow(imagen, cmap = “Greys”)
    plt.show()

from random import sample

for i in sample(list(range(len(X_train))), 3):
    titulo = "Mostrando imagen X_train[" + str(i) + "]"
    titulo = titulo + " -- Y_train[" + str(i) + "] = " + str(Y_train[i])
    show_image(X_train[i], titulo)
```

Gráficas de los resultados con Matplotlib

Puedes probar más cosas para adquirir destreza en Python con Numpy y Matplotlib, o sacar conclusiones acerca de los datos sobre los que trabajamos. Podrías contar la cantidad de imágenes de cada categoría en el conjunto de entrenamiento, o mostrar por pantalla los primeros tres perros. Por ejemplo, el siguiente código imprime una gráfica con los valores de las etiquetas de las 20 primeras imágenes del conjunto de test:

```
def plot_curva(Y, titulo, xscale = "linear", yscale = "linear"):
    plt.title(titulo)
    plt.plot(Y)
    plt.xscale(xscale)
    plt.yscale(yscale)
    plt.show()

plot_X(Y_test[:20], "Etiquetas de los primeros 20 valores")
```

Para los ejercicios y tareas de esta práctica deberás crear varias funciones que te facilitarán el trabajo y reducirán el tamaño del código. En la plantilla de código en Moodle ya se incluye una función `cargar_cifar10()` que devuelve los cuatro conjuntos de datos de Cifar 10, otras dos funciones para entrenar redes neuronales, una que realiza el entrenamiento con repeticiones de un modelo de MLP (definido por los los parámetros que se le pasen) con el conjunto de entrenamiento y la evaluación de los resultados con el conjunto de test y otra que permite entrenar diferentes modelos de MLP y mostrar los resultados para comparar, y otras dos funciones que emplean matplotlib para mostrar las gráficas que facilitarán el análisis de rendimiento de los modelos a partir de las predicciones del modelo y las clases verdaderas, una que dibuja la evolución de la pérdida y la tasa de acierto para el conjunto de entrenamiento y para el conjunto de validación durante el entrenamiento de la red (en Keras, esta información la devuelve la función *fit*, que es la que realiza el entrenamiento), y otra que dibuja la gráfica de barras con el tiempo de entrenamiento y la tasa de acierto finales con el conjunto de test (en Keras, esto lo conseguimos con la función *evaluate*) de varios modelos, para comparar resultados de varios modelos. Podrás modificar esas funciones como te sea necesario o crear otras. Por ejemplo, haría falta crear una función para dibujar la matriz de confusión para los resultados con el conjunto de test (investiga cómo puedes mostrar esa información de manera rápida y elegante, quizás tengas

que instalar alguna otra librería de Python).

3) Primera parte: aprender a configurar redes neuronales con MLPs de Keras

> **Recuerda**: Entrega de la primera parte hasta el **29 de noviembre de 2026 a las 23:55h**.
>
> La **no entrega** de este hito supone hasta un **20% de penalización** en la nota final de esta práctica.

3.1 Definir, entrenar y evaluar un MLP con Keras, e identificar el número adecuado de épocas para evitar sobre- y subentrenamiento.

El **sobreentrenamiento** ocurre cuando la red aprende el ruido del conjunto de entrenamiento y pierde rendimiento ante muestras con las que no se ha entrenado. Y viceversa, el **subentrenamiento** ocurre cuando la red no tiene suficiente tiempo para aprender las características útiles del conjunto de entrenamiento y no alcanza el máximo rendimiento ni con el conjunto de entrenamiento ni con otras muestras. En este primer ejercicio aprenderás a declarar y emplear un MLP de Keras y a ajustar cuánto entrenarlo para maximizar su rendimiento.

En un MLP se utilizan capas Dense (es decir, cada neurona de una capa está conectada a todas las neuronas de la anterior capa), que no tienen en cuenta la organización espacial de las características de una observación (en nuestro caso, de una imagen de Cifar10) y que esperarán que todas las características de una misma muestra aparezcan en un único vector (es decir, los datos deben *aplanarse* antes de entrar a una capa Dense).

En la tarea de Moodle tienes una plantilla con el código necesario para entrenar este primer MLP, que explicaremos en la primera sesión. Esta plantilla ya incluye un par de funciones que emplean la librería matplotlib para mostrar los resultados del entrenamiento y la evaluación: una para dibujar la evolución de la tasa de acierto y el error durante el entrenamiento mediante curvas, y otra para comparar el rendimiento de diferentes modelos con una gráfica de barras. En este apartado utilizaremos la primera para identificar sobre- o subentrenamiento. Eres libre de modificar esas funciones como te sea necesario.

Ten en cuenta que el entrenamiento de estos modelos parte de estados iniciados al azar, que influyen en el éxito del entrenamiento, por lo que dos entrenamientos diferentes del mismo modelo (todos los parámetros iguales) darán resultados diferentes, a veces muy diferentes. Por ello es necesario realizar varios entrenamientos independientes y analizar los resultados promedio, e idealmente descartar resultados *outlier*. En esta práctica sobra con que hagas tres repeticiones de cada entrenamiento (guardas cada *history*, los promedias al final y eso es lo que dibujas), pero ten en cuenta que para conseguir robustez estadística en un entorno profesional se necesitarían más (o muchas más) repeticiones, dependiendo de la complejidad del problema y de lo bien ajustado que estén el resto de parámetros para evitar quedar atrapado en mínimos locales demasiado pronto.

Ejercicio 1: entrenar un MLP sencillo y observar su evolución

Realiza un entrenamiento con un número de épocas elevado (100) para visualizar la evolución de la tasa de acierto de los conjuntos de entrenamiento y de validación, y selecciona el valor de época adecuado para maximizar la tasa de acierto con el conjunto de validación.

Queda a tu criterio documentarte sobre otras técnicas de control de las épocas, como el callback EarlyStopping (https://keras.io/api/callbacks/).

3.2 Algunos parámetros básicos de un MLP

Al entrenar un MLP (y cualquier tipo de red neuronal), además del número de épocas existen una serie de parámetros que controlan el proceso del entrenamiento. En este apartado tratamos dos de los más básicos:

Batch Size (Tamaño de lote): Es la cantidad de ejemplos de entrenamiento que se procesan simultáneamente en una sola iteración antes de actualizar los pesos de la red. Un tamaño pequeño introduce más ruido, un entrenamiento más errático, pero puede ayudar a salir de mínimos locales; uno grande permite un entrenamiento más suave y aprovecha mejor la paralelización (por lo que es mucho más rápido) pero requiere más memoria para almacenar las activaciones antes de aplicar backpropagation.

Learning Rate (Tasa de aprendizaje): Es el factor que determina el tamaño del paso que da el optimizador al actualizar los pesos en dirección al gradiente descendente. Si es muy alto, el entrenamiento oscila fuertemente o diverge; si es muy bajo, el aprendizaje puede volverse extremadamente lento o estancarse.

Ejercicio 2: batch_size y learning_rate

Entrena varios MLP de 2 capas de 128 neuronas con activación Sigmoid, variando los valores de batch size (se recomienda que sean múltiplos de 2 para optimizar el uso de memoria y de la GPU), y muestra los resultados en gráficas de barras para comparar el rendimiento de cada modelo. Analiza los efectos que observes para obtener conclusiones generales que te sirvan de guía en los ejercicios finales. Haz lo mismo con el learning rate (por ejemplo para valores entre 0.01 y 0.0001).

3.3 Funciones de activación y profundidad de la red

El rendimiento de un MLP depende directamente de la relación entre la función de activación seleccionada (puedes consultar las disponibles en Keras aquí: https://keras.io/api/layers/activations/) y la profundidad de la red (número de capas), debido a las propiedades matemáticas de la función de activación y cómo se propaga el gradiente a través de la red:

Funciones simétricas o acotadas alrededor de cero (con GlorotNormal):

Sigmoid: Sufre de desvanecimiento del gradiente (saturación en sus extremos, donde la derivada es casi 0). En redes de más de 3 capas, las primeras capas dejan de aprender. Solo recomendada en redes muy poco produndas (hasta 2 capas). Se recomienda inicializar las entradas al rango [0, 1].

TanH: Está centrada en cero, lo que acelera la convergencia respecto a Sigmoid, pero sigue sufriendo saturación y desvanecimiento de gradiente en arquitecturas profundas. Permite entrenar redes poco profundas (con 4 capas aún funciona bastante bien). Se recomienda inicializar las entradas al rango [-1, 1].

Funciones rectificadoras (con HeNormal):

ReLU: Su derivada es constante (1) para entradas positivas, lo que evita el desvanecimiento del gradiente. Sin embargo, en arquitecturas extremadamente profundas puede sufrir el problema del "ReLU muerto" (neuronas con salida 0 permanente). Mantiene el rendimiento en redes profundas (hasta 10 capas). Se recomienda inicializar las entradas al rango [-2, 2].

LeakyReLU: Al permitir una pequeña pendiente para valores negativos, evita la muerte de neuronas y permite entrenar MLPs más profundos con mayor estabilidad que ReLU. Mantiene el rendimiento en redes de mayor profundidad que ReLU. Se recomienda inicializar las entradas al rango [-2, 2].

GELU: Curvatura suave y estocástica que combina propiedades de regularización. Mantiene un flujo continuo del gradiente a lo largo de profundidades elevadas. Mantiene el rendimiento en redes de gran profundidad (50+ capas). Se recomienda inicializar las entradas al rango [-2, 2].

Función autonormalizable (con LecunNormal):

SELU: Auto-normaliza la media y la varianza de las activaciones en cada capa hacia 0 y 1 (es necesario usar la inicialización LecunNormal), garantizando la estabilidad del gradiente en redes muy profundas sin necesidad de Batch Normalization. Mantiene el rendimiento en redes de muy gran profundidad (100+ capas). Se recomienda inicializar las entradas al rango [-2, 2].

Funciones suaves (con HeNormal):

Swish/SiLU: Sus gradientes continuos facilitan la optimización en redes profundas, superando habitualmente a ReLU a mayor profundidad. Fueron optimizadas para redes con conexiones residuales. Se recomienda inicializar las entradas al rango [-2, 2].

Mish: Similar a Swish pero con un perfil de curvatura más suave. Preserva un flujo de gradiente altamente regularizado a través de múltiples capas. Mantiene el rendimiento en redes de gran profundidad. Se recomienda inicializar las entradas al rango [-2, 2].

Ejercicio 3: funciones de activación, inicializadores de pesos y escalado inicial

Ahora has de entrenar MLPs con diferente número de capas (por ejemplo 2, 4 y 8), todas las capas de 128 neuronas, utilizando diferentes funciones de activación para ver su rendimiento en redes de distinta profundidad. Recomedamos probar con al menos una función acotada (Sigmoid o TanH), una función rectificadora (ReLU, LeakyReLU y GELU) y con la función autonormalizadora (SELU). Las funciones suaves (Mish y Swish) pueden dar también muy buenos resultados aunque no estén ideadas para redes poco profundas como las que vamos a probar aquí.

Muestra en una gráfica la tasa de acierto y el tiempo de entrenamiento para cada MLP y analiza qué modelo es más interesante y de qué formas se podría mejorar el rendimiento con cada función de activación.

3.4 Técnicas de normalización: Batch Normalization

Batch Normalization (BN) es una técnica utilizada en redes neuronales profundas para normalizar las activaciones de una capa intermedia restando la media del mini-lote actual y dividiéndola por su desviación estándar. Posteriormente, aplica una escala y un desplazamiento aprendibles. Sus objetivos principales son:

- Reducir el desplazamiento de covariables interno (internal covariate shift).
- Estabilizar y acelerar el entrenamiento.
- Permitir el uso de tasas de aprendizaje más altas y actuar como un regulizador ligero.

Normalmente se recomienda aplicar BN a la salida de cada capa oculta antes de aplicar la activación, y no incluirla tras la última capa oculta (antes de la capa de salida).

BN interactúa de manera distinta según el tipo de función de activación elegida, la profundidad de la red y el valor de batch size. En particular, no se recomienda su uso con redes poco profundas, ya que el ruido estocástico que introduce supera al beneficio de la regularización; además, necesita un batch size mínimo para que la media y varianza sean mínimamente representativas de la muestra completa. Además, no deben utilizarse en redes con SELU ya que destruyen la autonormalización, y su uso con funciones suaves como Mish o Swish se debe considerar según la profundidad de la red y el problema a resolver.

Queda a tu criterio documentarte sobre otras técnicas de normalización, como Layer Normalization (https://keras.io/api/layers/normalization_layers/).

Ejercicio 4: batch normalization

Selecciona unos cuantos de los MLP del ejercicio 3 para aplicarles BN y analiza los efectos.

3.5 Técnicas de regularización: Dropout.

En redes neuronales, la regularización se refiere a cualquier técnica diseñada para prevenir el sobreajuste (overfitting). Durante la fase de entrenamiento, Dropout desconecta una fracción de las neuronas poniendo sus activaciones a cero, seleccionando aleatoriamente en cada paso de actualización las neuronas a desactivar.

Los objetivos principales de Dropout son:

- Prevenir el sobreentrenamiento (overfitting), al dificultar que la red se fije en el ruido del conjunto de entrenamiento.
- Evitar la coadaptación excesiva entre neuronas vecinas y obligar a la red a aprender representaciones redundantes y robustas.

Queda a tu criterio documentarte sobre otras técnicas de regularización, como por ejemplo el decaimiento L2 (https://keras.io/api/layers/regularization_layers/).

Ejercicio 5: dropout

De nuevo, selecciona alguno de los MLP de los ejercicios 3 (sin BN) y 4 (con BN) para aplicarles Dropout con distintos valores (por ejemplo entre 0.2 y 0.5), y analiza los resultados. Ten en cuenta que puede haber interacciones positivas o negativas al combinar diferentes técnicas, o que la colocación ideal de unas capas con respecto a otras puede variar en función de qué técnicas mezcles.

4) Segunda parte: desarrollar un modelo eficiente con CNN de Keras

Recuerda: Entrega final hasta el **23 de diciembre de 2026 a las 23:55h**.

En este apartado desarrollarás una arquitectura mucho más potente que un MLP a la hora de clasificar imágenes gracias a emplear redes convolucionales. cuyas “neuronas” (o unidades) aplican filtros de convolución (https://en.wikipedia.org/wiki/Kernel_(image_processing)) a las imágenes de entrada, permitiendo la extracción de características de más alto nivel, por lo que están mejor adaptadas a problemas de visión artificial.

4.1 Definir, entrenar y evaluar un CNN sencillo con Keras

Antes de empezar con la optimización de tu red neuronal final, vamos a realizar un pequeño ejercicio como toma de contacto con las CNN. Para completar este ejercicio necesitarás documentarte sobre las CNN y buscar ejemplos de implementación con Keras. Encontrarás varios en la propia web de Keras (p.ej. https://keras.io/examples/vision/mnist_convnet/). Ten en cuenta que las capas de una CNN utilizan la información espacial y de color, por lo que las imágenes no se deben aplanar como requieren las capas Dense de un MLP: las capas Conv2D deben recibir la imagen con su distribución espacial intacta. Una novedad importante es que utilizarás capas de pooling para reducir la complejidad del problema.

Ejercicio 6: CNN básica con Maxpooling

Para empezar a experimentar, declara una CNN con dos bloques de convolución compuestos cada uno de una capa Conv2D con filtros (*kernel*) de 3x3 y activación ReLU, seguida de una capa de MaxPooling2D con filtros de 2x2. El primer bloque ha de tener 16 filtros de convolución y el segundo 32. Al final de la red debes añadir una capa oculta Dense de 128 neuronas, y finalmente, como capa de salida otra capa Dense con función de activación softmax. Deja el resto de parámetros en sus valores por defecto pero ajusta *epochs* como hiciste en

ejercicios anteriores (si utilizas *EarlyStopping* ten en cuenta que puede requerir unos parámetros muy diferentes a los que iban bien en los ejercicios con MLP).

Queda a tu criterio documentarte sobre otras técnicas de pooling (https://keras.io/api/layers/pooling_layers/).

4.2 Tarea final: desarrollar una CNN para Cifar10 con optimización libre

Tu objetivo ahora es optimizar la arquitectura de una CNN para maximizar su tasa de acierto empleando unos recursos limitados: No debes utilizar más de 32 filtros en la primera capa Conv2D, y debes utilizar *MaxPooling2D* u otra técnica de pooling como mínimo después de las primera capa Conv2D. Además, ninguna capa Conv2D puede tener más de 128 filtros. En cuanto a las capas Dense que puedes añadir (o no) al final, no debes utilizar más de 256 neuronas en la primera ni más de 128 en las siguientes. Estas restricciones son para evitar que experimentes con arquitecturas muy lentas y pierdas mucho tiempo que necesitas para aprender otras cosas.

Las mejoras que propongas deberán estar fundamentadas, implementadas y evaluadas convenientemente para que puedan ser tenidas en consideración. Ten en cuenta que ciertas técnicas no serán efectivas al mezclarlas con otras, o que necesitarán de otras técnicas para extraer todo su potencial. Cuantas más mejoras pruebes y documentes (aunque no ayuden a mejorar la tasa de acierto final) más nota tendrás en esta tarea, pero en general se puede alcanzar la nota máxima probando con tres de las mejora listadas arriba.

Es recomendable que te documentes antes de iniciar la experimentación, para apoyarte en resultados previos de otros investigadores (todo bien referenciado en tu bibliografía) y poder dirigir mejor tus tiempo y esfuerzos. Pero ten en cuenta que en muchos de los modelos que podrás encontrar en la literatura del arte no tienen esa limitación de número de filtros: no puedes utilizar los modelos grandes tal cuál porque tardarán demasiado, pero sí puedes sacar ideas de su arquitectura y las técnicas auxiliares empleadas.

Se valora que el código esté correctamente comentado y que sea autoexplicativo (evitar nombres de funciones o variables que no se entiendan). No incluyas fragmentos de código en la documentación salvo que sea imprescindible para explicar algo y que creas que no es suficiente con comentarlo en el código.

Una vez realizado el estudio y la experimentación con los modelos de MLP que decidas, debes documentar bien todo tu trabajo, mostrando las gráficas relevantes y comentando los resultados. Para comprender mejor los resultados de los modelos generados, se deberán incluir matrices de confusión que permitan visualizar las categorías sobre las que se confunde.

Finalmente, deberás indicar cuál es la mejor configuración obtenida, comparando los resultados en forma de tabla resumen, justificando adecuadamente dicha decisión, y desarrollando en tus conclusiones por qué crees que unas arquitecturas son mejores que otras.

5) Consideraciones finales

- En el hito intermedio deberás entregar la parte de los MLP, como mínimo con todas las tareas completadas, y lo que puedas de la documentación, antes del 29 de noviembre de 2026 a las 23:55h.
- Esta entrega intermedia no se evalúa, pero la no entrega de este hito, o una entrega incompleta, tendrá una penalización de hasta el 20% de la nota final de la práctica.
- En la entrega final deberás entregar la práctica completa y tendrás opción de mejorar lo que hubieras entregado en el hito intermedio.
- El seguimiento durante las sesiones supone el 40% de la nota y la entrega final el 60%.
- Las prácticas son individuales, no se pueden hacer en grupos y no se puede reusar código o texto de otros estudiantes ni de otras fuentes sin referenciarlo. La entrega se realizará **a través de Moodle**, y

debe consistir en dos ficheros, **un fichero .pdf con la documentación y otro fichero .py con todo el código Python**, tanto las implementaciones de los algoritmos requeridos como el código utilizado para producir todos los resultados que muestres en la documentación (gráficas, tablas, etc.).

- No se puede entregar la práctica en varios ficheros de Python. Si deseas programar en varios archivos por el motivo que sea, asegúrate de juntarlo todo en un único archivo .py y comprobar que todo funciona correctamente antes de la entrega.
- Los dos archivos de la entrega se deben llamar con el nombre completo del estudiante, usando '_' como separador entre palabras (p.ej. Juan_Carlos_Sánchez-Arjona_de_los_Rios.py y .pdf). En el caso de que creas necesario adjuntar algún otro archivo (una hoja de cálculo con resultados, un archivo de log, etc.) consulta primero a tu profesor/a.

AVISO IMPORTANTE

No cumplir cualquiera de las normas de entrega descritas al principio de este documento puede suponer un suspenso de la práctica.

Las prácticas son individuales. No se pueden hacer en pareja o grupos.

Cualquier código o texto copiado supondrá un suspenso de la práctica para todos los estudiantes implicados y **se informará a la dirección de la EPS para la toma de medidas oportunas** (Reglamento para la Evaluación de Aprendizajes de la Universidad de Alicante, BOUA 9/12/2015, y documento de Actuación ante copia en pruebas de evaluación de la EPS).